

IN3050/IN4050 Mandatory Assignment 3: Unsupervised Learning

Name: Justas Simutis

Username: justassi

Rules

Before you begin the exercise, review the rules at this website:

<https://www.uio.no/english/studies/examinations/compulsory-activities/mn-ifi-mandatory.html>

, in particular the paragraph on cooperation. This is an individual assignment. You are not allowed to deliver together or copy/share source-code/answers with others. Read also the "Routines for handling suspicion of cheating and attempted cheating at the University of Oslo":

<https://www.uio.no/english/studies/examinations/cheating/index.html> We do not entirely prohibit the use of generative language models ("smart assistants" like ChatGPT, Llama, Claude or Copilot), but you must clearly acknowledge this at all times, following the UiO guidelines:

https://www.uio.no/english/studies/resources/ai_student.html Note also that you must fully understand *all* the parts of your submissions, even if you got some help from a generative model.

This will be tested during your peer review sessions

(<https://www.uio.no/studier/emner/matnat/ifi/IN3050/v25/Peer%20review/>). By submitting this assignment, you confirm that you are familiar with the rules and the consequences of breaking them.

Delivery

Deadline: Tuesday, April 22, 2025, 23:59 CEST

Your submission should be delivered in Devilry. You may make several submissions in Devilry before the deadline, but only the last delivery will be read, so make sure to include all files in the last delivery. It is recommended to upload preliminary versions hours (or days) before the final deadline.

What to deliver?

We recommend you to solve all exercises in this Jupyter notebook (alternative 1), but it is also possible to work with a regular Python script (alternative 2).

Alternative 1: If you choose Jupyter, you should deliver the notebook. You should answer all questions and explain what you are doing in Markdown. Still, the code should be properly commented. The notebook should contain results of your runs. In addition, you also have to submit a PDF of your solution which shows the results of the runs.

Alternative 2: If you prefer not to use notebooks, you should deliver the code as Python scripts together with your run results and a PDF report where you answer all the questions and explain your work.

Here is a list of *absolutely necessary* (but not sufficient) conditions to get the assignment marked as passed:

- You must deliver your code (Python script or Jupyter notebook) you used to solve the assignment.
- The code used for making the output and plots must be included in the assignment.
- You must include example runs that clearly shows how to run all implemented functions and methods.
- All the code (in notebook cells or python main-blocks) must be runnable. If you have unfinished code that crashes, please comment it out and document what you think causes it to crash.
- You must also deliver a PDF of the code, outputs, comments and plots as explained above.

Make sure to include your name and username in the submitted files. Deliver one single compressed folder (.zip, .tgz or .tar.gz) which contains your complete solution.

Important: if you weren't able to finish the assignment, use the PDF report/Markdown to elaborate on what you've tried and what problems you encountered. Students who have made an effort and attempted all parts of the assignment will get a second chance even if they fail initially. This exercise will be graded PASS/FAIL.

Goals of the assignment

This assignment has two parts:

1. You will go through the basic theory of **Principal Component Analysis (PCA)** and implement PCA from scratch to compress and visualize data.
2. You will run **K-means clustering** using the `scikit-learn` toolkit and use PCA to visualize the results. You will also evaluate the output of K-means using a multi-class logistic regression classifier.

IN4050 students will have to do one extra part about tuning PCA to balance compression with information lost.

Tools

You may freely use code from the weekly exercises and the published solutions. In the first part about PCA you may **NOT** use ML libraries like `scikit-learn`. In the K-means part and beyond we encourage the use of `scikit-learn` to iterate quickly on the problems.

We will use the `numpy` library for performing matrix computations and the `pyplot` library for plotting data, as well as `scikit-learn` for K-means clustering. This assignment also comes with a module called `data_assignment3` that you will use to import different (synthetic and real) datasets. Let's start by making sure that everything is installed.

```
%pip install numpy matplotlib scikit-learn
import data_assignment3
```

```
Requirement already satisfied: numpy in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (1.26.4)
Requirement already satisfied: matplotlib in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (3.10.0)
Requirement already satisfied: scikit-learn in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (1.5.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from matplotlib) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from matplotlib) (11.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from matplotlib) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: scipy>=1.6.0 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from scikit-learn) (1.15.1)
Requirement already satisfied: joblib>=1.2.0 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: six>=1.5 in c:\users\justa\anaconda3\envs\in3050\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

Part 1: Principal Component Analysis (PCA)

In this section, you will work with the PCA algorithm in order to understand its definition and explore its uses.

Part 1.1: Implementation

Here we implement the basic steps of PCA and we assemble them. We will only need functions from *numpy* for this part.

```
import numpy as np
```

Centering the Data

Implement a function with the following signature to center the data. Remember that every *feature* should be centered.

```
def center_data(A):  
    # INPUT:  
    # A      [NxM] numpy data matrix (N samples, M features)  
    #  
    # OUTPUT:  
    # X      [NxM] numpy centered data matrix (N samples, M features)  
  
    # Implementation  
    mean = A.mean(axis=0) # Getting the vertical mean  
    result = A - mean     # Offsetting the data  
  
    # Returning result  
    return result
```

Test your function checking the following assertion on *testcase* (absence of output means that the assertion holds):

```
testcase = np.array([[3., 11., 4.3], [4., 5., 4.3], [5., 17., 4.5],  
                    [4, 13., 4.4]])  
answer = np.array([[-1., -0.5, -0.075], [0., -6.5, -0.075], [1., 5.5,  
                    0.125], [0., 1.5, 0.025]])  
np.testing.assert_array_almost_equal(center_data(testcase), answer)
```

Computing Covariance Matrix

Implement a function with the following signature to compute the covariance matrix. In order to get this at the correct scale, divide by $N - 1$, not N .

Note: Numpy provides a function `np.cov()` that does exactly this, but in this exercise we ask you to implement the code from scratch without using this function.

```
def compute_covariance_matrix(A):  
    # INPUT:  
    # A      [NxM] numpy data matrix (N samples, M features)  
    #  
    # OUTPUT:  
    # C      [MxM] numpy covariance matrix (M features, M features)  
  
    # Setup  
    n = A.shape[0] # Counting samples  
    centered = center_data(A) # Centering data (Accounting for mean  
    in the formula)  
  
    # Calculating
```

```

    result = centered.T @ centered # Dot product (Accounting for
matrix multiplication and summation in the formula)
    result = result / (n - 1)      # Dividing by sample size (Also
using Bessel's correction)

# Returning result
return result

```

Test your function by comparing its output to the output of `np.cov()`:

```

test_array = np.array([[22., 11., 5.5], [10., 5., 2.5], [34., 17., 8.5],
[28., 14., 7]])
answer = np.cov(np.transpose(test_array))
to_test = compute_covariance_matrix(test_array)
np.testing.assert_array_almost_equal(to_test, answer)

```

Computing eigenvalues and eigenvectors

Use the linear algebra package of `numpy` and its function `np.linalg.eig()` to compute eigenvalues and eigenvectors. Note that we only take the real part of the eigenvectors and eigenvalues. The covariance matrix *should* be a symmetric matrix, but the actual implementation in `compute_covariance_matrix()` can lead to small round off errors that lead to tiny imaginary additions to the eigenvalues and eigenvectors. These are purely numerical artifacts that we can safely remove.

Note: If you decide to NOT use `np.linalg.eig()` you must make sure that the eigenvalues you compute are of unit length!

```

def compute_eigenvalue_eigenvectors(A):
    # INPUT:
    # A      [DxD] numpy matrix
    #
    # OUTPUT:
    # eigval  [D] numpy vector of eigenvalues
    # eigvec  [DxD] numpy array of eigenvectors

    eigval, eigvec = np.linalg.eig(A)

    # Numerical roundoff can lead to (tiny) imaginary parts. We
correct that here.
    eigval = eigval.real
    eigvec = eigvec.real

    return eigval, eigvec

```

Test your function checking the following assertion on *testcase*:

```

testcase = np.array([[2, 0, 0], [0, 5, 0], [0, 0, 4]])
answer_eigval = np.array([2., 5., 4.])
answer_eigvec = np.array([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]])
x, y = compute_eigenvalue_eigenvectors(testcase)
np.testing.assert_array_almost_equal(x, answer_eigval)
np.testing.assert_array_almost_equal(y, answer_eigvec)

```

Sorting eigenvalues and eigenvectors

Implement a function with the following signature to sort eigenvalues and eigenvectors in descending order.

Remember that eigenvalue `eigval[i]` corresponds to eigenvector `eigvec[:, i]`.

```

def sort_eigenvalue_eigenvectors(eigval, eigvec):
    # INPUT:
    # eigval      [D] numpy vector of eigenvalues
    # eigvec      [DxD] numpy array of eigenvectors
    #
    # OUTPUT:
    # sorted_eigval      [D] numpy vector of eigenvalues
    # sorted_eigvec      [DxD] numpy array of eigenvectors

    sorted_eigval, sorted_eigvec = [], []

    # Getting sorted indexes (Descending order)
    indexes = np.argsort(eigval)[::-1]

    # Sorting vector columns
    sorted_eigvec = eigvec[:, indexes]

    # Sorting the eigen values
    for n in indexes:
        sorted_eigval.append(eigval[n])

    # Converting to numpy arrays
    sorted_eigval = np.array(sorted_eigval)
    sorted_eigvec = np.array(sorted_eigvec)

    return sorted_eigval, sorted_eigvec

```

Test your function checking the following assertion on *testcase*:

```

testcase = np.array([[2, 0, 0], [0, 5, 0], [0, 0, 4]])
answer_eigval = np.array([5., 4., 2.])
answer_eigvec = np.array([[0., 0., 1.], [1., 0., 0.], [0., 1., 0.]])
x, y = compute_eigenvalue_eigenvectors(testcase)
x, y = sort_eigenvalue_eigenvectors(x, y)

```

```
np.testing.assert_array_almost_equal(x, answer_eigval)
np.testing.assert_array_almost_equal(y, answer_eigvec)
```

PCA Algorithm

Implement a function with the following signature to compute PCA using the functions implemented above.

```
def pca(A, m):
    # INPUT:
    # A      [NxM] numpy data matrix (N samples, M features)
    # m      integer number denoting the number of learned features (m
    <= M)
    #
    # OUTPUT:
    # pca_eigvec [Mxm] numpy matrix containing the eigenvectors (M
    dimensions, m eigenvectors)
    # P         [Nxm] numpy PCA data matrix (N samples, m
    features)

    # Calculating PCA eigen vectors
    pca_eigvec = compute_covariance_matrix(A)
    pca_eigvec = compute_eigenvalue_eigenvectors(pca_eigvec)
    # Getting eigen values and vectors
    pca_eigvec = sort_eigenvalue_eigenvectors(pca_eigvec[0],
    pca_eigvec[1]) # Sorting eigen values and vectors
    pca_eigvec = pca_eigvec[1][:, :m]
    # Keeping only eigen vectors with m dimensions

    # Computing P with eigen vectors
    P = center_data(A) @ pca_eigvec

    # Printing result
    return pca_eigvec, P
```

Test your function checking the following assertion on *testcase*:

```
import pickle
testcase = np.array([[22., 11., 5.5], [10., 5., 2.5], [34., 17.,
8.5]])
x, y = pca(testcase, 2)

answer1 = pickle.load(open('PCAanswer1.pkl', 'rb'))
answer2 = pickle.load(open('PCAanswer2.pkl', 'rb'))
test_arr_x = np.sum(np.abs(np.abs(x) - np.abs(answer1)), axis=0)
np.testing.assert_array_almost_equal(test_arr_x, np.zeros(2))
test_arr_y = np.sum(np.abs(np.abs(y) - np.abs(answer2)))
np.testing.assert_almost_equal(test_arr_y, 0)
```

Part 1.2: Understanding - How does PCA work?

We now use the PCA algorithm you implemented on a toy data set in order to understand its inner workings.

Loading the data

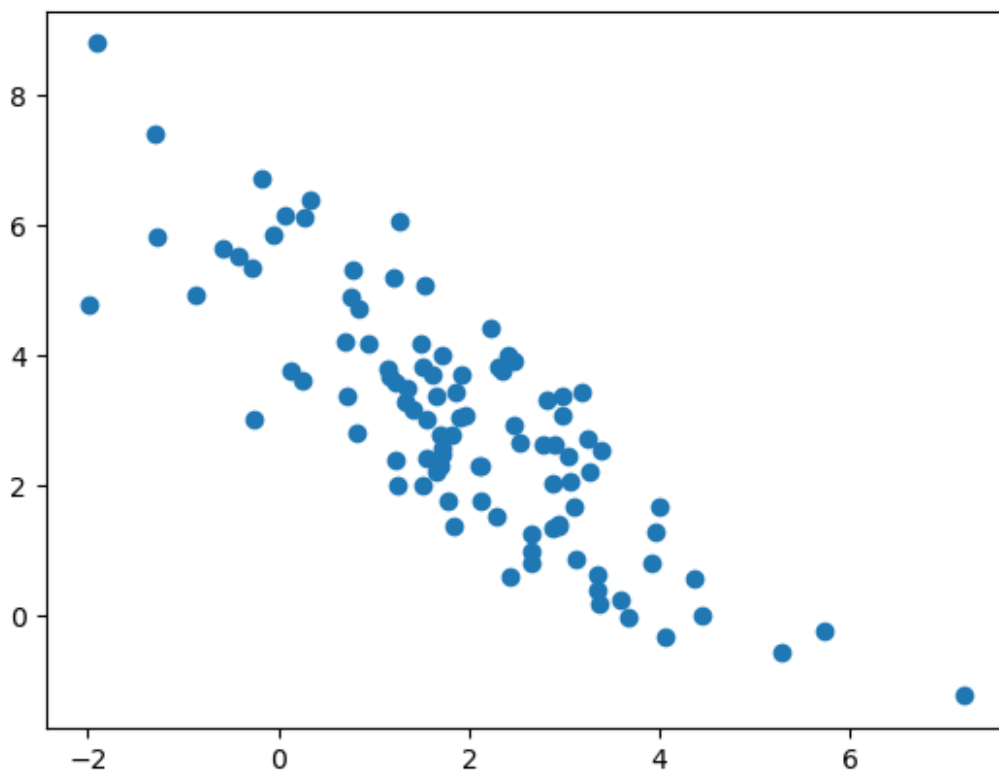
The module `data_assignment3` provides a small synthetic dataset of dimension $[100 \times 2]$ (100 samples, 2 features):

```
import data_assignment3  
  
X = data_assignment3.get_synthetic_data()
```

Visualizing the data

Visualize the synthetic data using the function `scatter()` from the `matplotlib` library.

```
import matplotlib.pyplot as plt  
plt.scatter(X[:, 0], X[:, 1])  
<matplotlib.collections.PathCollection at 0x19ec0944da0>
```



Visualize the centered data

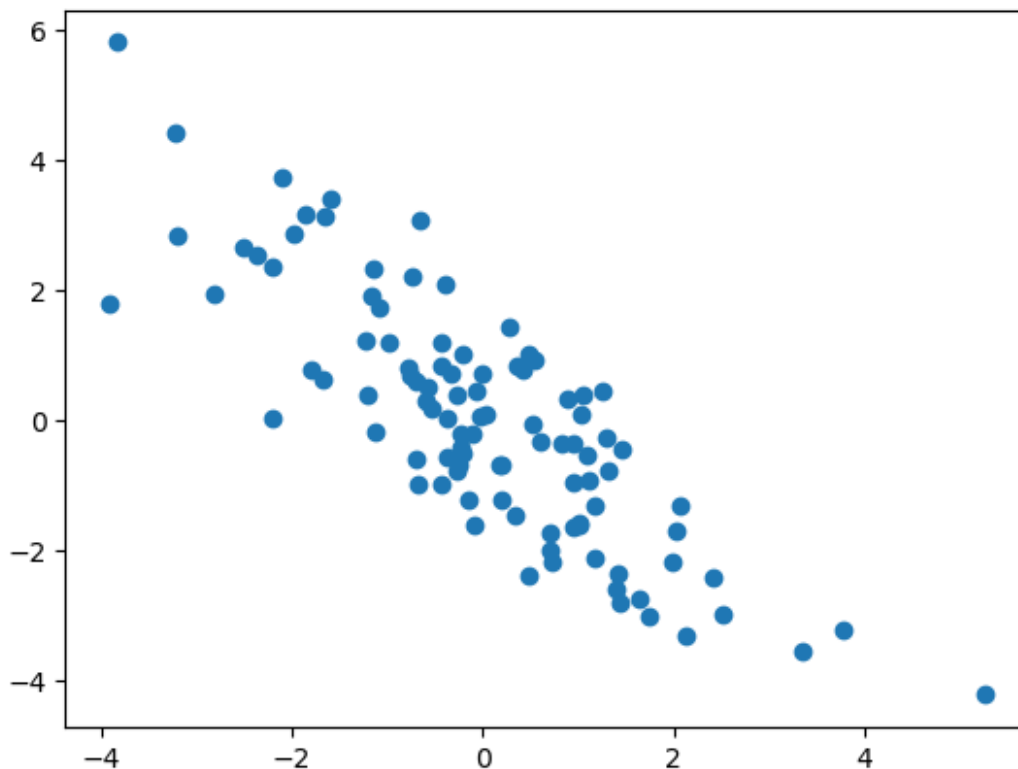
Notice that the data visualized above is not centered on the origin (0,0). Use the function defined above to center the data, and the replot it.

```
## Your code here

# Centering the data
X_0 = center_data(X) # X_0 because X is now centered at Origin or 0
for short

# Plotting data
plt.scatter(X_0[:, 0], X_0[:, 1])

<matplotlib.collections.PathCollection at 0x19ec09d8dd0>
```



Visualize the first eigenvector

Visualize the vector defined by the first eigenvector. To do this you need:

- Use the `PCA()` function to recover the eigenvectors
- Plot the centered data as done above
- The first eigenvector is a 2D vector (x_0, y_0) . This defines a vector with origin in $(0,0)$ and head in (x_0, y_0) . Use the function `plot()` from matplotlib to plot a line over the first eigenvector.

```

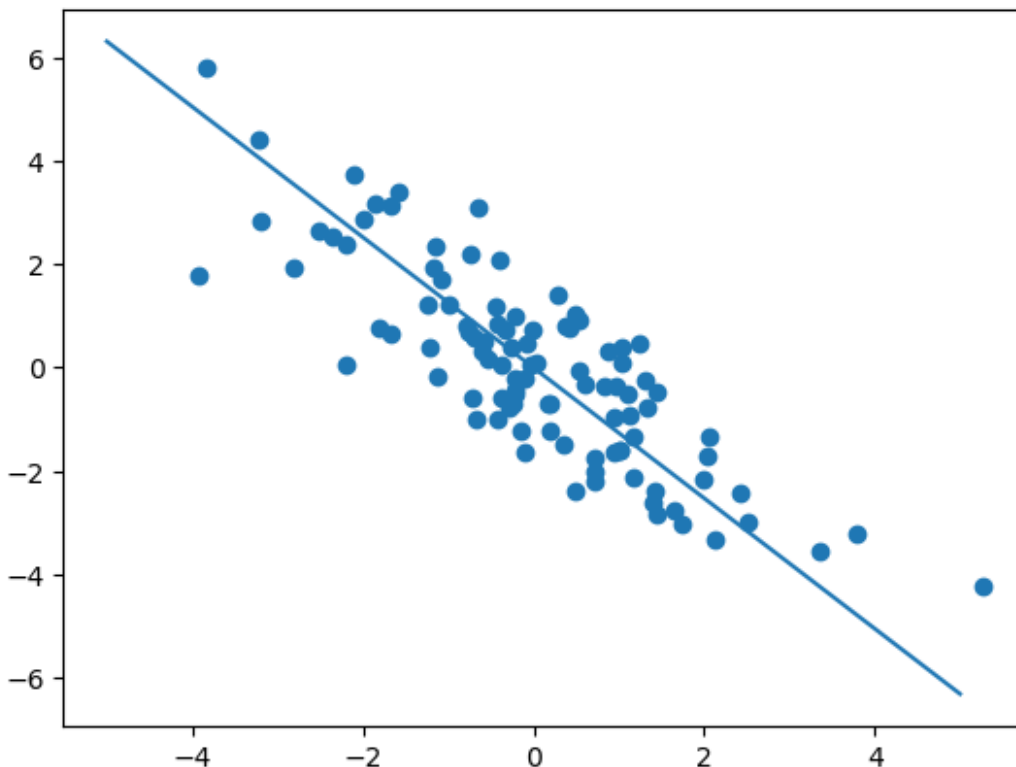
pca_eigvec = pca(X, 2)[0]      # m=2 Because 2 dimensions, [0]
# because we want only the eigen vector
first_eigvec = pca_eigvec[0]   # Self explanatory

plt.scatter(X_0[:, 0], X_0[:, 1]) # Plotting centered data

x = np.linspace(-5, 5, 1000)
y = first_eigvec[1]/first_eigvec[0] * x
plt.plot(x, y)

[<matplotlib.lines.Line2D at 0x19ec0ba8d10>]

```



Visualize the PCA projection

Finally, use the `PCA()` algorithm to project on a single dimension and visualize the result using again the `scatter()` function.

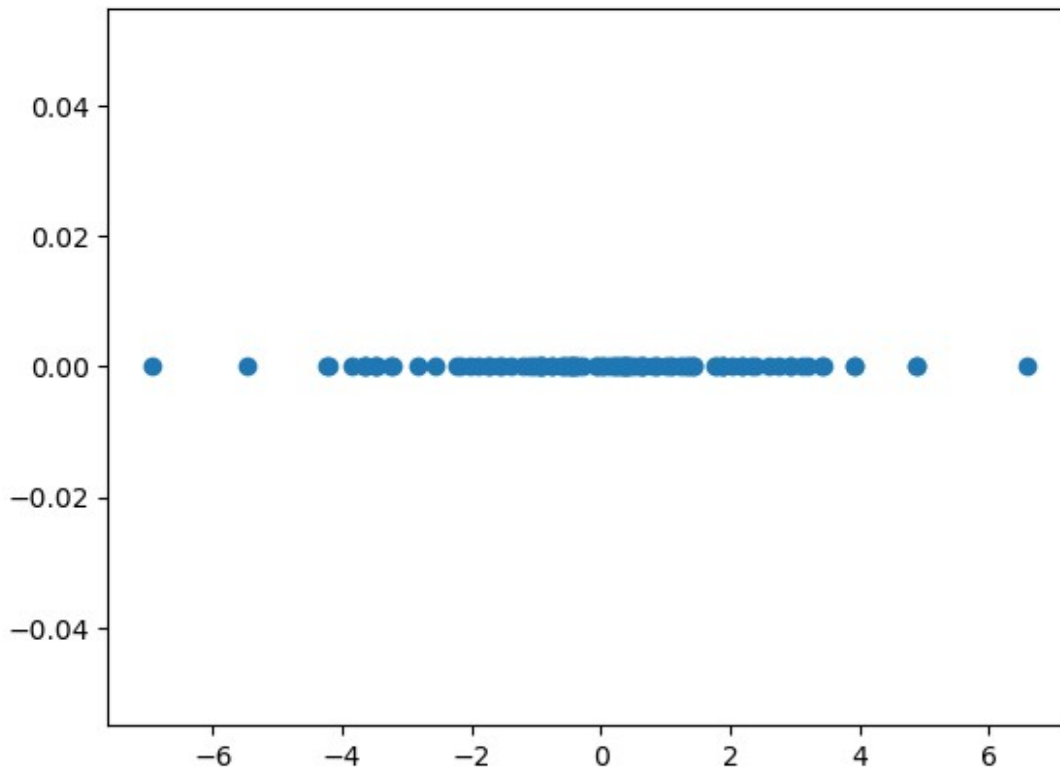
```

# This time we keep the 1 dimensional pca projection of X
result = pca(X, 1)[1]

# Plotting result
plt.scatter(result, np.zeros(len(result)))

<matplotlib.collections.PathCollection at 0x19ec0966e10>

```



Part 1.3: Evaluation - When are the results of PCA sensible?

So far we have used PCA on synthetic data. Let us now imagine we are using PCA as a pre-processing step before a classification task. This is a common setup with high-dimensional data. We explore when the use of PCA is sensible.

Loading the first set of labels

The function `get_synthetic_labeled_data_1()` from the module `data_assignment3` provides a first labeled dataset.

```
X, y = data.assignment3.get_synthetic_labeled_data(1)
```

Running PCA

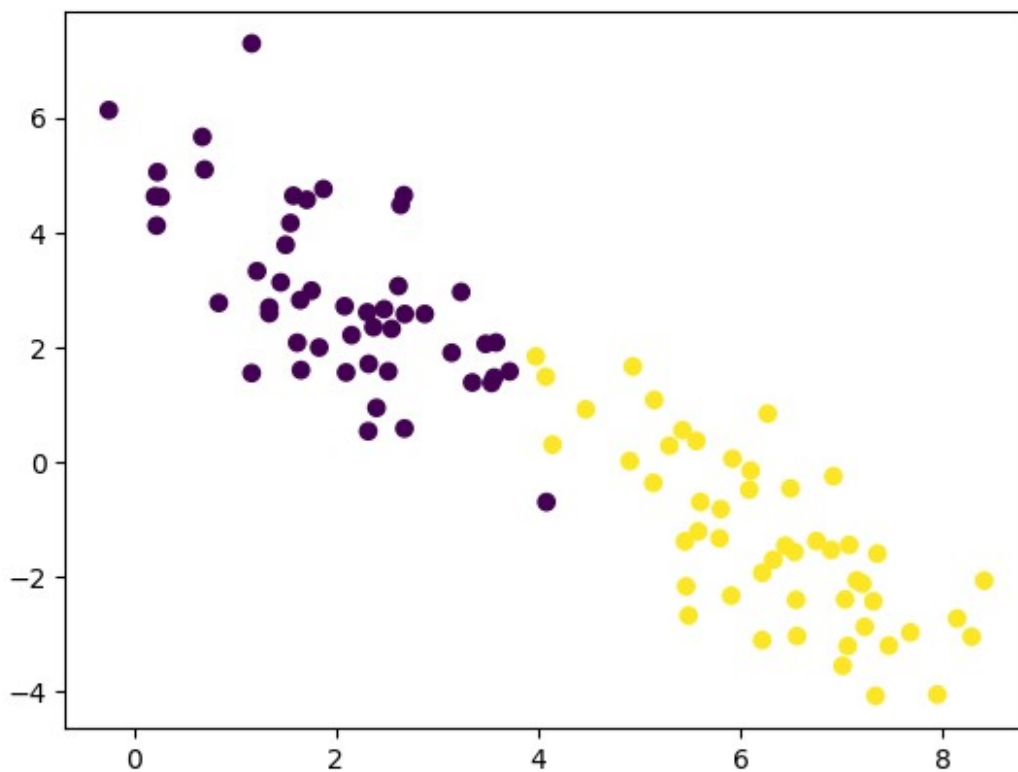
Process the data using the PCA algorithm and project it in one dimension. Plot the labeled data using `scatter()` before and after running PCA. Comment on the results.

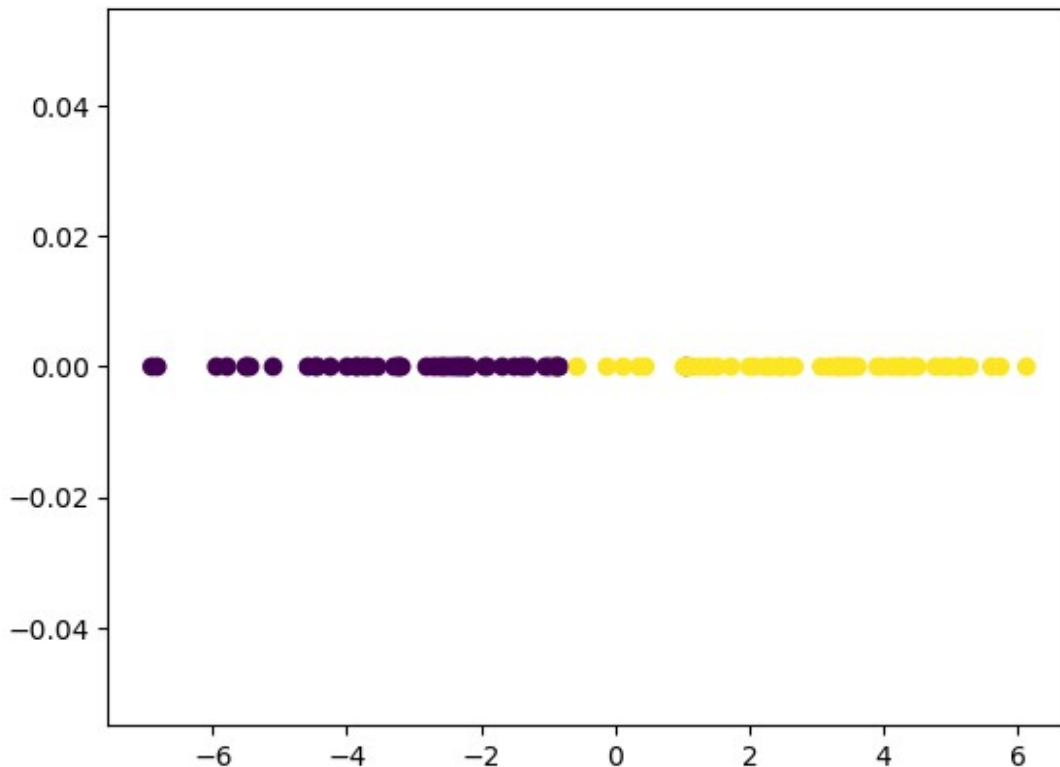
```
# before PCA
plt.scatter(X[:, 0], X[:, 1], c=y[:, 0])

# after PCA
plt.figure() # Splitting plots
```

```
result = pca(X, 1)[1] # Getting projection
plt.scatter(result, np.zeros(len(X)), c=y) # Projecting projection
(haha, get it?)
```

```
<matplotlib.collections.PathCollection at 0x19ec0bc9b80>
```





Comment: I looked at the last purple points on the left and one more emerged there from top part and yeah, that makes sense. I also noticed how bottom right yellow part of the cluster had lots on the left side, but only 1 is left there which also makes sense since the elevation and height matters and they are sorta far from each other when it comes to this aspect. The middle is less cramped which is preserved. The left and right middles are more populated as seen in the clusters in 2D as well. Overall, great job me :D

Loading the second set of labels

The function `get_synthetic_labeled_data_2()` from the module `data_assignment3` provides a second labeled dataset.

```
X, y = data_assignment3.get_synthetic_labeled_data_2()
```

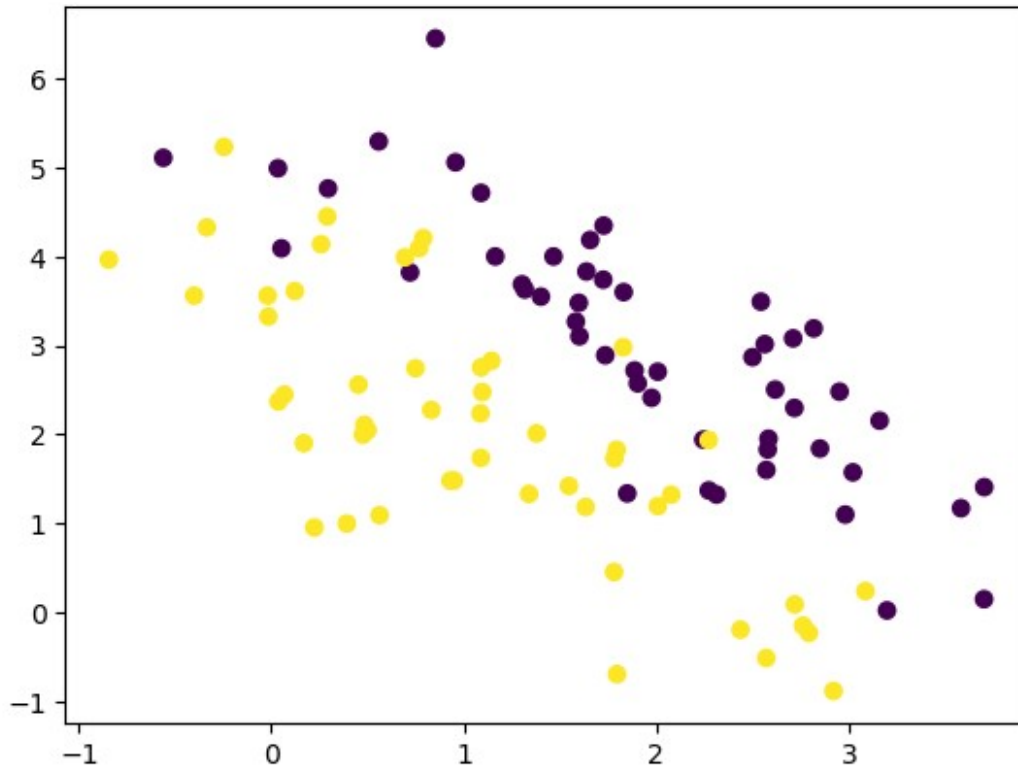
Running PCA

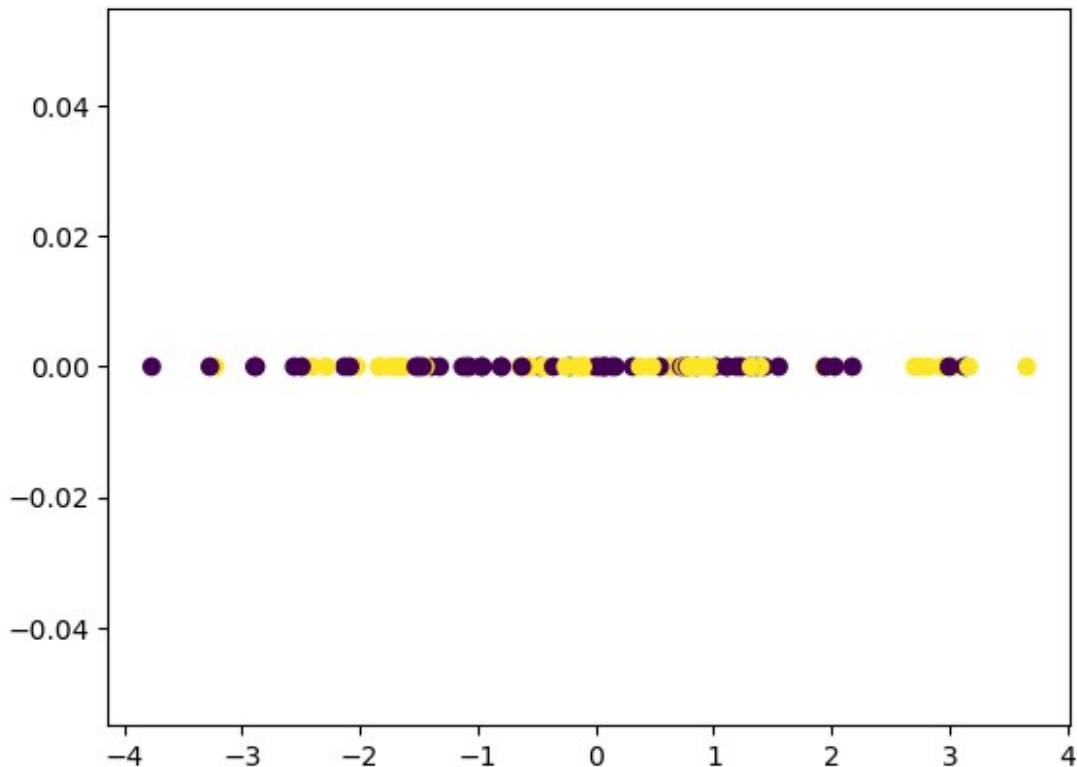
As before, process the data using the PCA algorithm and project it in one dimension. Plot the labeled data using `scatter()` before and after running PCA. Comment on the results.

```
# before PCA
plt.scatter(X[:, 0], X[:, 1], c=y[:, 0])

# after PCA
plt.figure()
```

```
result = pca(X, 1)[1]
plt.scatter(result, np.zeros(len(X)), c=y)
<matplotlib.collections.PathCollection at 0x19ec0b18920>
```





Comment: In this situation, the data looks chaotic, unorganized, and not clustered. Yet that is because of our point of view. This is a great example of one of the downsides of such approach. This can be easily fixed by looking at the data from other perspectives, other eigen vectors.

How would the result change if you were to consider only the second eigenvector? What about if you were to consider both eigenvectors?

Answer: If I were to consider the second eigenvector the data would look much more structured, organized, clustered. If I were to look at both separately I could get a very good understanding of it all. But having many more than just two could prove difficult to manage and comprehend the scale/structure of.

Part 1.4: Case study 1 - PCA for visualization

The [iris flower dataset](#) is one of the oldest and best known data collections used for machine learning. It consists of 50 samples from each of three species of iris flowers (*Iris setosa*, *Iris virginica* and *Iris versicolor*). Four features were measured from each sample: sepal length, sepal width, petal length and petal width, all in centimeters.

Visualizing a 4-dimensional dataset is impossible; therefore we will use PCA to project our data in 2 dimensions and visualize it.

Loading the data

The function `get_iris_data()` from the module `data_assignment3` returns the *iris* dataset. It returns a data matrix of dimension [150x4] and a label vector of dimension [150].

```
X, y = data_assignment3.get_iris_data()
```

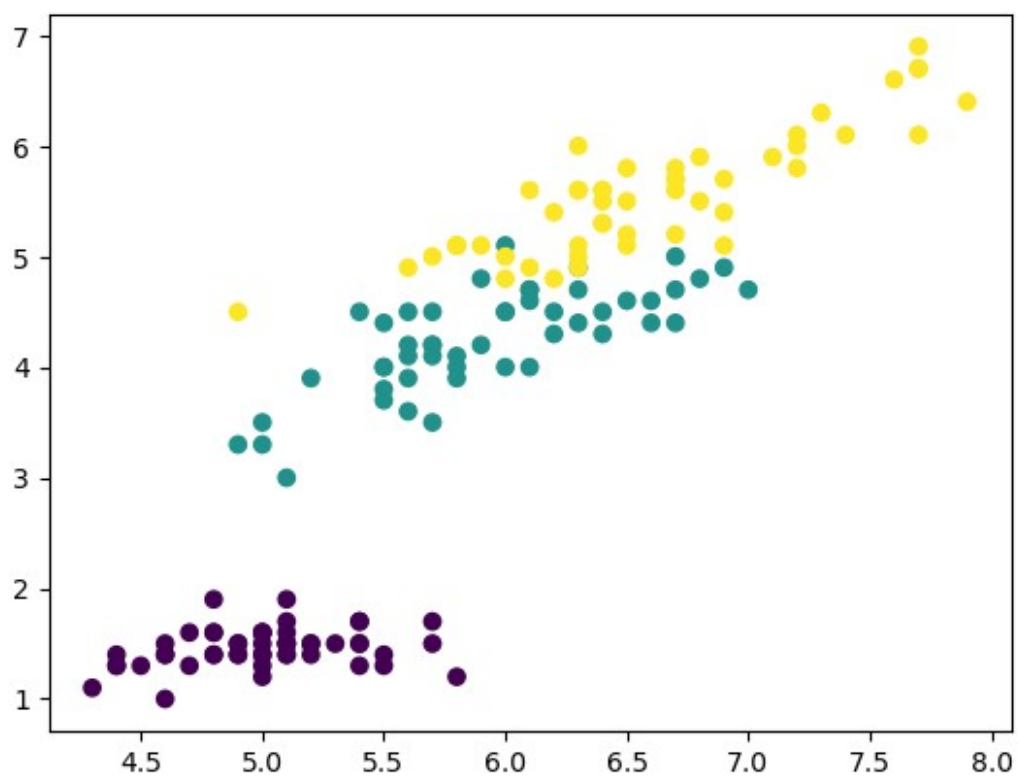
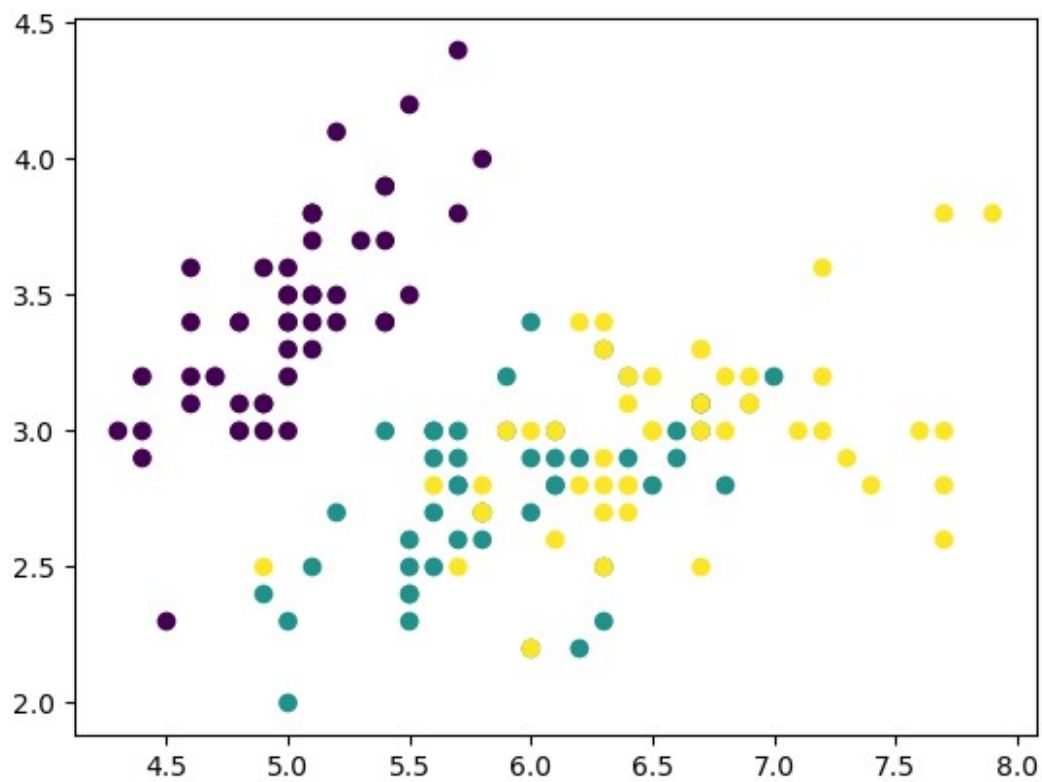
Visualizing the data by selecting features

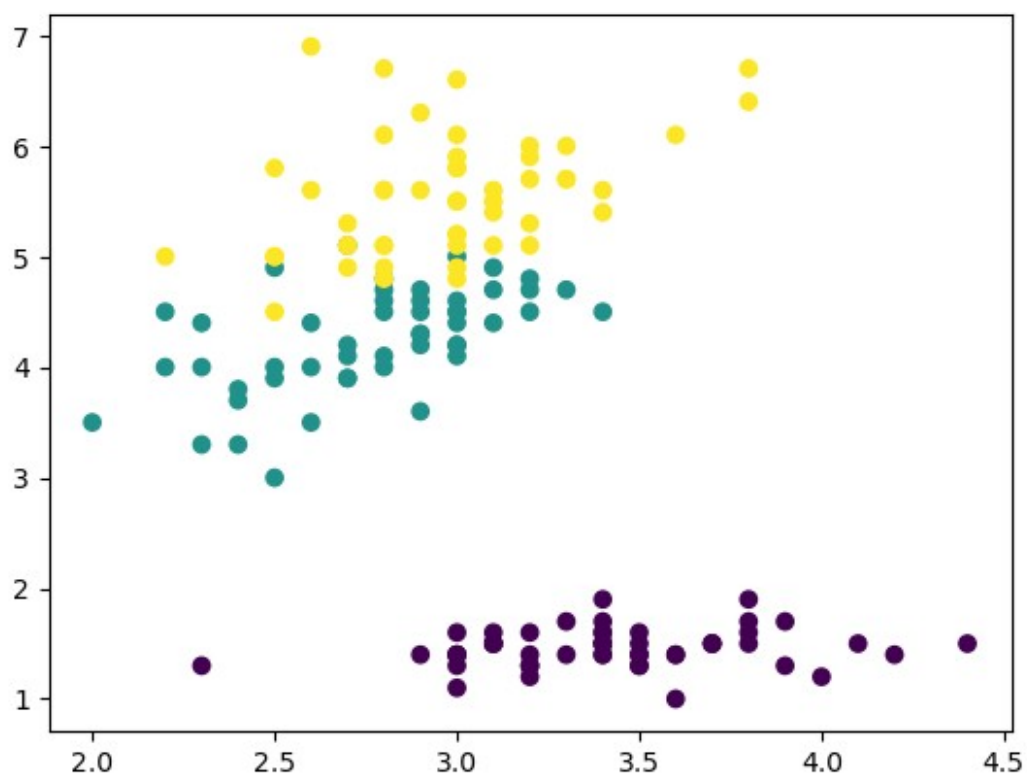
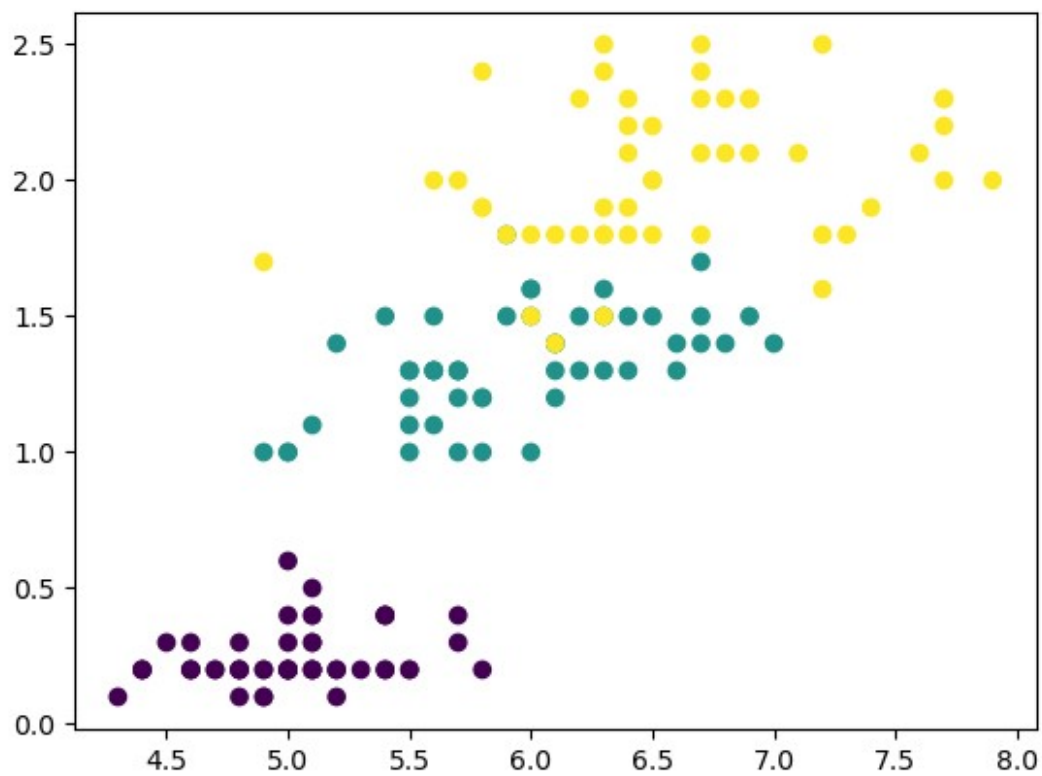
Try to visualize the data (using label information) by randomly selecting two out of the four features of the data. You may try different pairs of features.

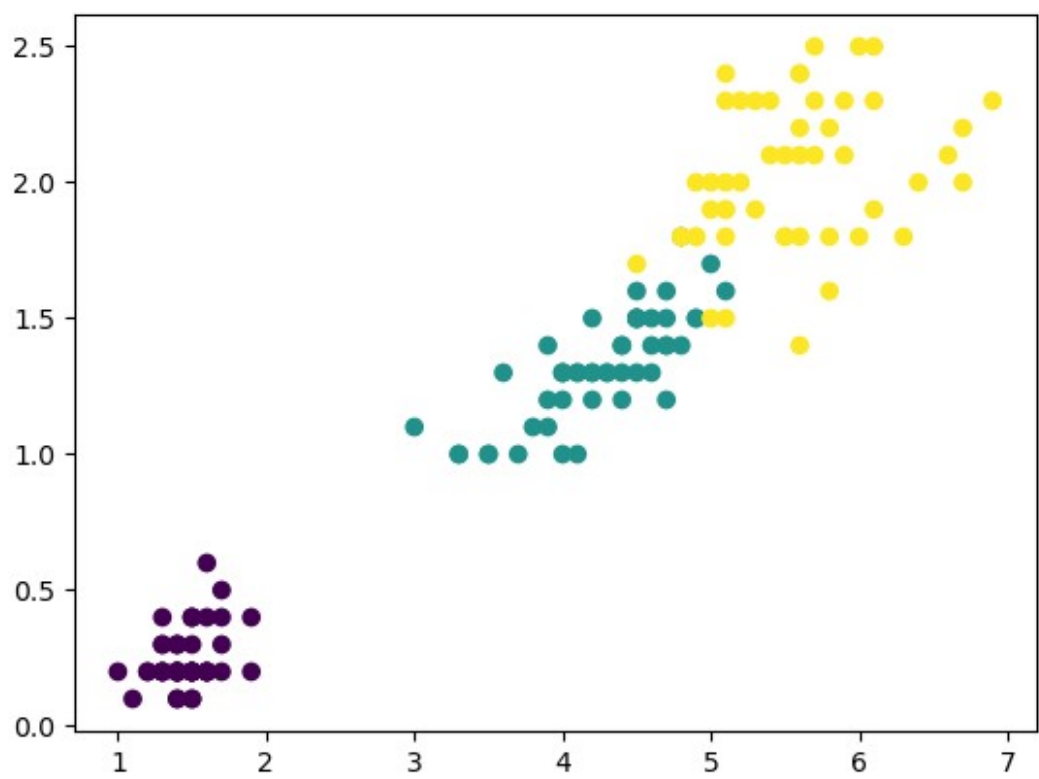
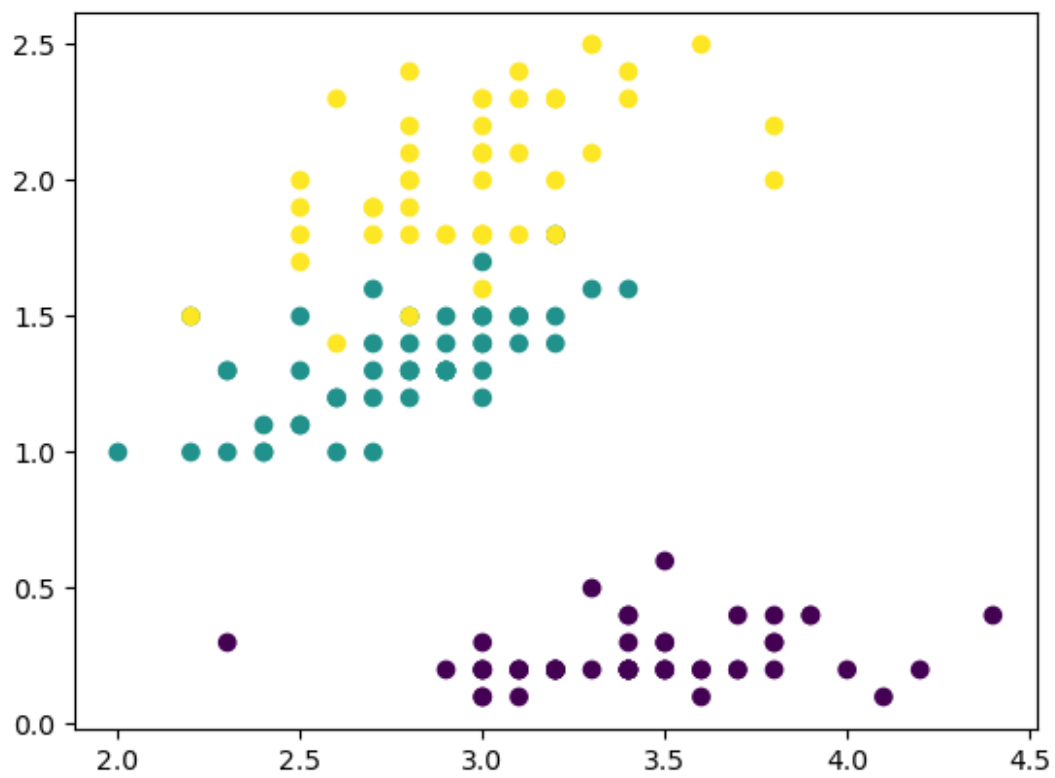
```
# Trying out different pairs to see how they compare
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.figure()
plt.scatter(X[:, 0], X[:, 2], c=y)
plt.figure()
plt.scatter(X[:, 0], X[:, 3], c=y)
plt.figure()
plt.scatter(X[:, 1], X[:, 2], c=y)
plt.figure()
plt.scatter(X[:, 1], X[:, 3], c=y)
plt.figure()
plt.scatter(X[:, 2], X[:, 3], c=y)

# As you can see that the purple flower type is different in all
plots.
# The cyan flower is in the middle, but closer to yellow in all but
one plot.
# Yellow seems to be the least clustered type with more variation in
every plot.
# This way we get a pretty clean picture of the flowers.

<matplotlib.collections.PathCollection at 0x19ec0b8c800>
```

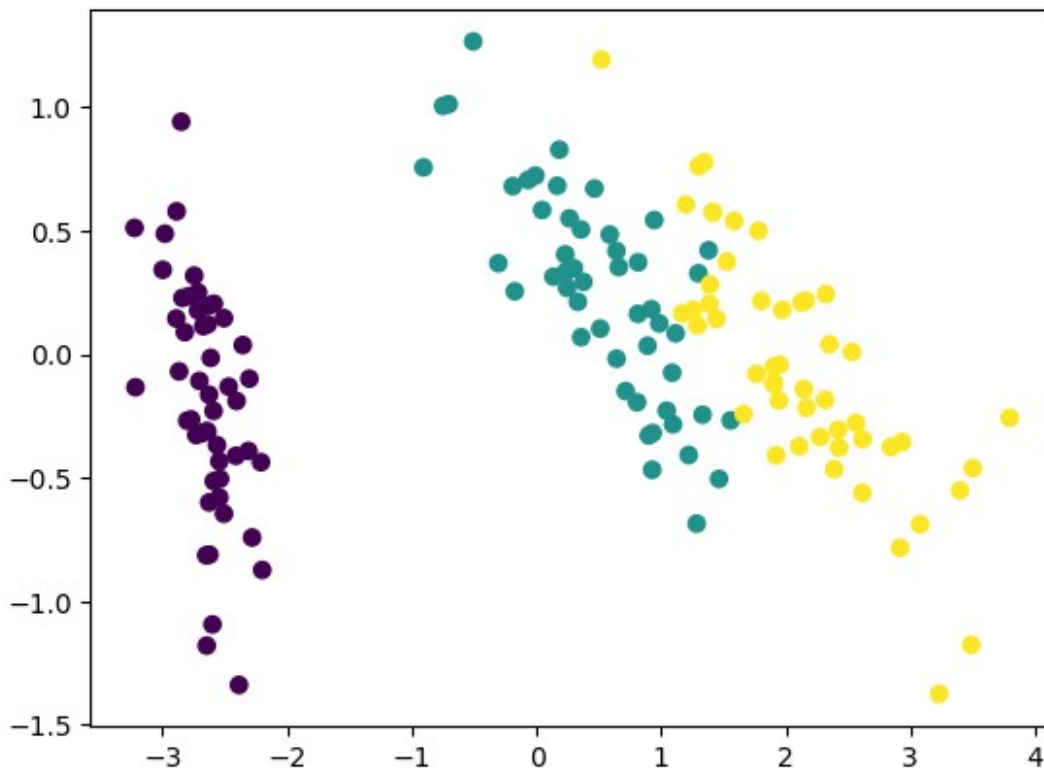




Visualizing the data by PCA

Process the data using PCA and visualize it (using label information). Compare with the previous visualization and comment on the results.

```
# Copy-Paste
result = pca(X, 2)[1]
plt.scatter(result[:, 0], result[:, 1], c=y)
<matplotlib.collections.PathCollection at 0x19ec0a99a00>
```



Comment: I've already described the non-pca plots, and this pca plot seems to get most of it. The purple is still shown as more extreme than the other two, or rather different. Cyan and yellow are close, but cyan is closer to the purple than yellow. Purple is shown as slightly more clustered when in reality it seems to be more clustered than that. Yellow is still perceived as more diversified than the other two. Not sure what else to point out.

Part 1.5: Case study 2 - PCA for compression

We now consider the *faces in the wild (lfw)* dataset, a collection of pictures (N=1280) of people. Each pixel in the image is a feature (M=2914).

Loading the data

The function `get_lfw_data()` from the module `data_assignment3` returns the *lfw* dataset. It returns a data matrix of dimension [1280x2914] and a label vector of dimension [1280]. It also returns two parameters, *h* and *w*, reporting the height and the width of the images (these parameters are necessary to plot the data samples as images). Beware, it might take some time to download the data. Be patient :)

```
X, y, h, w = data_assignment3.get_lfw_data()
```

Inspecting the data

Choose one datapoint to visualize (first coordinate of the matrix *X*) and use the function `imshow()` to plot and inspect some of the pictures.

Notice that the first argument of *imshow* is the image to be plotted; the image must be provided as a rectangular matrix, therefore we reshape a sample from the matrix *X* to have height *h* and width *w*. The parameter *cmap* specifies the color coding; in our case we will visualize the image in black-and-white with different gradations of grey.

```
image_number = 4      # display the 4th image of the collection
plt.imshow(X[image_number, :].reshape((h, w)), cmap=plt.cm.gray)
<matplotlib.image.AxesImage at 0x19ec250f2f0>
```



Implementing a compression-decompression function

Implement a function that first uses PCA to project samples in low-dimensions, and then reconstruct the original image.

Hint: Most of the code is the same as the previous PCA() function you implemented.

```
def encode_decode_pca(A, m):  
    # INPUT:  
    # A      [NxM] numpy data matrix (N samples, M features)  
    # m      integer number denoting the number of learned features (m  
    <= M)  
    #  
    # OUTPUT:  
    # Ahat [NxM] numpy PCA reconstructed data matrix (N samples, M  
    features)  
  
    # Saving mean for reconstruction  
    original_mean = A.mean(axis=0)  
  
    # Applying PCA  
    projection, eigenvectors = pca(A, m)  
  
    # Reverting changes  
    Ahat = (projection @ eigenvectors.T).T + original_mean  
  
    return Ahat
```

Compressing and decompressing the data

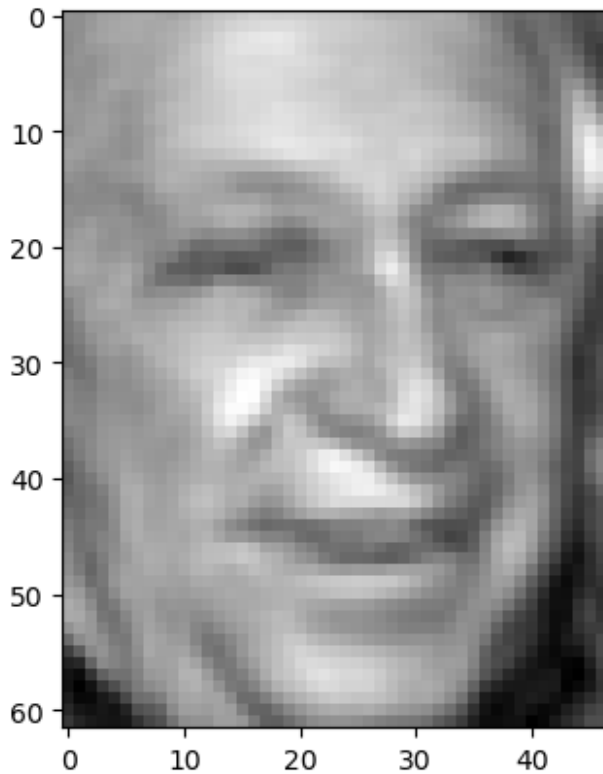
Use the implemented function to encode and decode the data by projecting on a lower dimensional space of dimension 200 (m=200).

```
Xhat = encode_decode_pca(X, 200)
```

Inspecting the reconstructed data

Use the function *imshow* to plot and compare original and reconstructed pictures. Comment on the results.

```
plt.imshow(Xhat[image_number, :].reshape((h, w)), cmap=plt.cm.gray)  
<matplotlib.image.AxesImage at 0x19ec0966ea0>
```



Comment: It looks more blurry. Most features are less visible, some are slightly incorrect like the outline of the bottom left cheek.

Evaluating different compressions

Use the previous setup to generate compressed images using different values of low dimensions in the PCA algorithm (e.g.: 20, 100, 200, 500, 1000). Plot and comment on the results. You can use `plt.subplot(n_rows, n_cols, position)` and `plt.title(titlestring)` to get a nice plot.

```
# List of dimensions to try out
dimension = (20, 100, 200, 500, 1000)
dimensions = len(dimension)

# Loop de loop
for i in range(dimensions):

    # Compressing and decompressing the image
    Xhat_m = encode_decode_pca(X, dimension[i])

    # Splitting plot
    plt.subplot(1, dimensions, i + 1)

    # Plotting reconstructed image
    plt.imshow(Xhat_m[image_number, :].reshape((h, w)),
               cmap=plt.cm.gray)
```

```
# Title
plt.title(f"{dimension[i]}")
plt.axis("off")
```



Comment: As the number of dimensions rises, so does the sharpness, or rather the resemblance to the original image. As we go down in dimensions we lose information which in turn makes it blurry since the data must be cramped into less dimensions, so some features are lost.

Part 1.6: PCA tuning (compulsory for Master students only)

If we use PCA for compression or decompression, it may be not trivial to decide how many dimensions to keep. In this section we review a principled way to decide how many dimensions to keep.

The number of dimensions to keep is the only *hyper-parameter* of PCA. A method designed to decide how many dimensions/eigenvectors is the *proportion of variance*:

$$POV = \frac{\sum_{i=1}^m \lambda_i}{\sum_{j=1}^M \lambda_j},$$

where λ are eigenvalues, M is the dimensionality of the original data, and m is the chosen lower dimensionality.

Using the *POV* formula we may select a number m of dimensions/eigenvalues so that the proportion of variance is, for instance, equal to 95%.

Implement a new PCA for encoding and decoding that receives in input not the number of dimensions for projection, but the amount of proportion of variance to be preserved.

```
def encode_decode_pca_with_pov(A, p):
    # INPUT:
    # A      [NxM] numpy data matrix (N samples, M features)
    # p      float number between 0 and 1 denoting the POV to be
    #         preserved
    #
    # OUTPUT:
```



```

    # Ahat [NxM] numpy PCA reconstructed data matrix (N samples, M
    features)
    # m    integer reporting the number of dimensions selected

    Ahat = None
    m = None
    return Ahat, m

```

Import the `lfw` dataset again. Use the implemented function to encode and decode the data by projecting on a lower dimensional space such that $POV=0.95$. Use the function `imshow` to plot and compare original and reconstructed pictures. Comment on the results.

```

X, y, h, w = data_assignment3.get_lfw_data()

Xhat, m = None, None
print("Selected dimensions:", m)

# Plot the images here

Selected dimensions: None

```

Comment: Enter your comment here.

Part 2: K-Means Clustering

In this section you will use the *k-means clustering* algorithm to perform unsupervised clustering. Then you will perform a qualitative assessment of the results.

Part 2.1: Applying K-Means and Qualitative Assessment

Importing scikit-learn library

We start importing the module `sklearn.cluster.KMeans` from the standard machine learning library `scikit-learn`.

```

from sklearn.cluster import KMeans

```

Loading the data

We will use once again the *iris* data set. Start by loading the dataset again.

```

X, y = data_assignment3.get_iris_data()

```

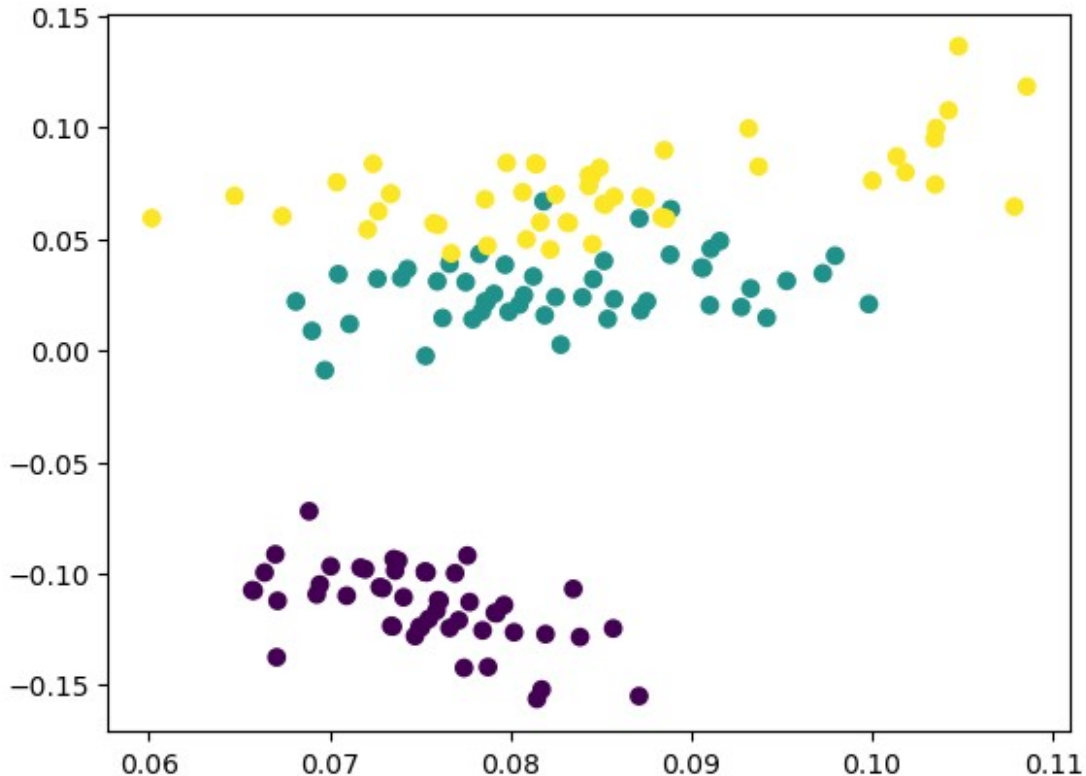
Projecting the data using PCA

To allow for visualization, we project our data in two dimensions as we did previously. This step is not necessary, and we may want to try to use *k-means* later without the PCA pre-processing. But to start, we use PCA, as this will allow for an easy visualization.

```
# Copy-paste with minor changes
projection, eigenvectors = pca(X.T, 2)
plt.scatter(projection[:, 0], projection[:, 1], c=y)

# Note to myself: We apply PCA before the KMeans mainly because it
reduces "noise".
# But it also speeds it up and makes it easier to visualize.
# Turns out KMeans is sensitive to outliers, so that is fixed as well.

<matplotlib.collections.PathCollection at 0x19ec0ae0500>
```



Running k-means

Use the class *KMeans* to fit and predict the output of the *k-means* algorithm on the projected data (note that we don't use the true labels *y* here). Run the algorithm using the following values of $k=\{2,3,4,5\}$.

```
# Please ignore the warnings.
```

```

k_values = [2, 3, 4, 5]
y_hats = []

# Random state to make results reproducible
seed = 42 # Classic, also an answer to "everything"

# Experimenting with K values
for i in range(len(k_values)):

    # Initializing KMeans algorithm
    kmeans = KMeans(n_clusters=k_values[i], random_state=seed)

    # Fitting data and storing results in a list
    y_hat = kmeans.fit_predict(projection)
    y_hats.append(y_hat)

    # P.S.
    # I don't like wearing hats ;(
    # It messes up my hair style

C:\Users\Justa\anaconda3\envs\in3050\Lib\site-packages\sklearn\
cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory
leak on Windows with MKL, when there are less chunks than available
threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\Justa\anaconda3\envs\in3050\Lib\site-packages\sklearn\
cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory
leak on Windows with MKL, when there are less chunks than available
threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\Justa\anaconda3\envs\in3050\Lib\site-packages\sklearn\
cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory
leak on Windows with MKL, when there are less chunks than available
threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\Justa\anaconda3\envs\in3050\Lib\site-packages\sklearn\
cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory
leak on Windows with MKL, when there are less chunks than available
threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.
  warnings.warn(

```

Qualitative assessment

Plot the results of running the k-means algorithm, compare with the true labels, and comment.

```

# Note:
# Checking if labels are correct is harder than one may initially
think.
# The labels may be correct, but the actual numbers be offset or mixed
up.
# The amount of combinations possible is {n!} where {n} is total label
count.

# I have implemented a function which I had done in Assignment 1
# It is different in a way that it looks cleaner :D

# Helper function to generate all permutations of a list
def generate_permutations(lst):
    if len(lst) == 0:
        return [[]]
    perms = []
    for i in range(len(lst)):
        rest = lst[:i] + lst[i+1:]
        for p in generate_permutations(rest):
            perms.append([lst[i]] + p)
    return perms

# Helper function to remap the list using a given permutation
def remap_list(original, mapping, unique_values):
    value_map = {unique_values[i]: mapping[i] for i in
range(len(unique_values))}
    return [value_map[val] for val in original]

# Helper function to generate all unique remapped versions
def generate_swapped_variants(original):
    unique_values = sorted(list(set(original)))
    all_perms = generate_permutations(unique_values)
    results = []
    seen = []

    for perm in all_perms:
        remapped = remap_list(original, perm, unique_values)
        if remapped not in seen:
            seen.append(remapped)
            results.append(remapped)

    return results

# Now the accuracy function itself
def accuracy(prediction, gold):

    # Taking shortest list length so we avoid IndexError
    (Not sure if that is what it's called but whatever)

```

```

list_len = min(len(prediction), len(gold))

# Getting all the unique variations of prediction
prediction_variations = generate_swapped_variants(prediction)
best_result = 0 # Also saving the best result so far

# Checking all variations
for variation in prediction_variations:

    # Counting how many were correct
    counter = 0
    for n in range(list_len):

        # Checking if gold
        if variation[n] == gold[n]:
            counter += 1

        # Ultra advanced mathematical formulas
        result = counter / list_len

    # Checking if it is the best we had so far
    if result > best_result:
        best_result = result

# Returning accuracy, range: from 0 to 1
# We also round to the nearest 4 decimals, looks cleaner
best_result = round(best_result, 4)
return best_result

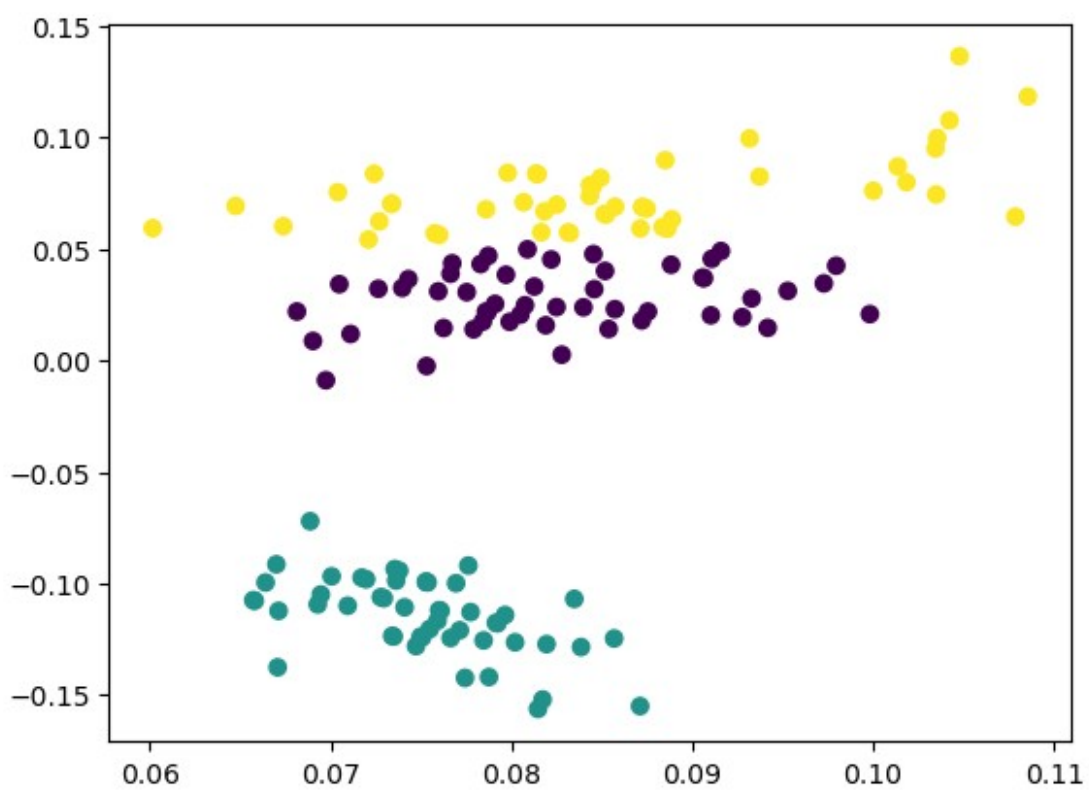
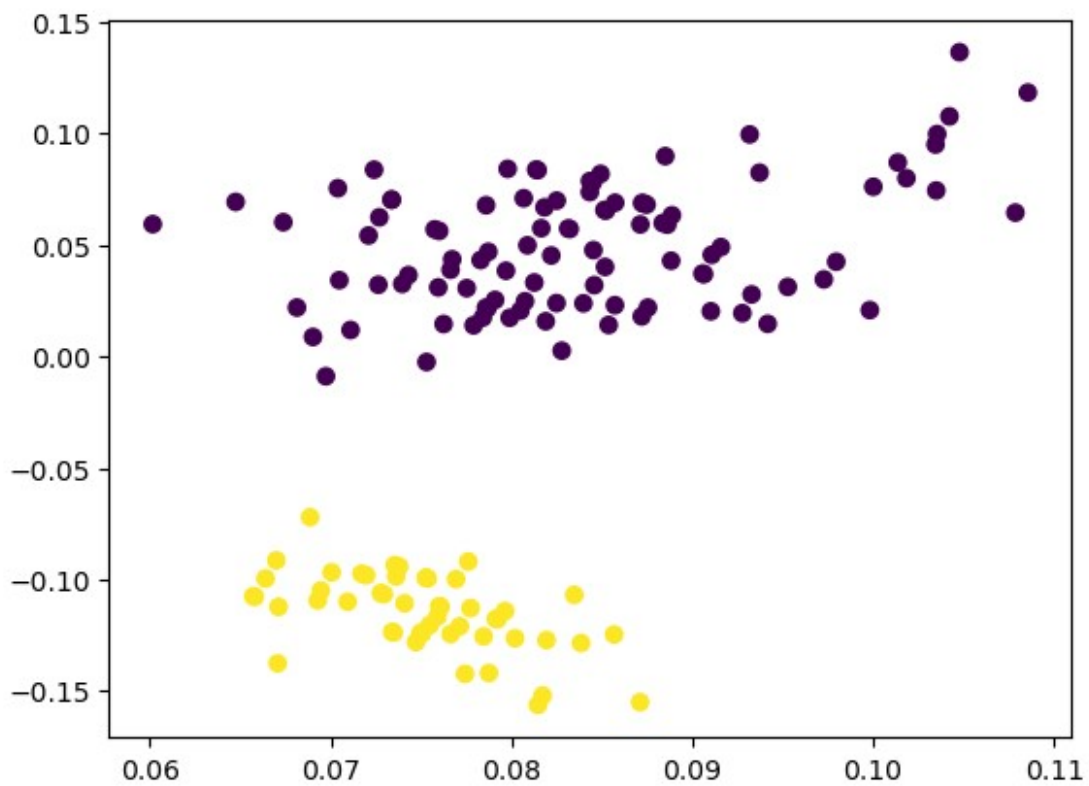
for hat in y_hats: # I named it a hat on purpose, that is because it
doesn't make any sense, but it may seem like it does
    plt.scatter(projection[:, 0], projection[:, 1], c=hat)
plt.figure()

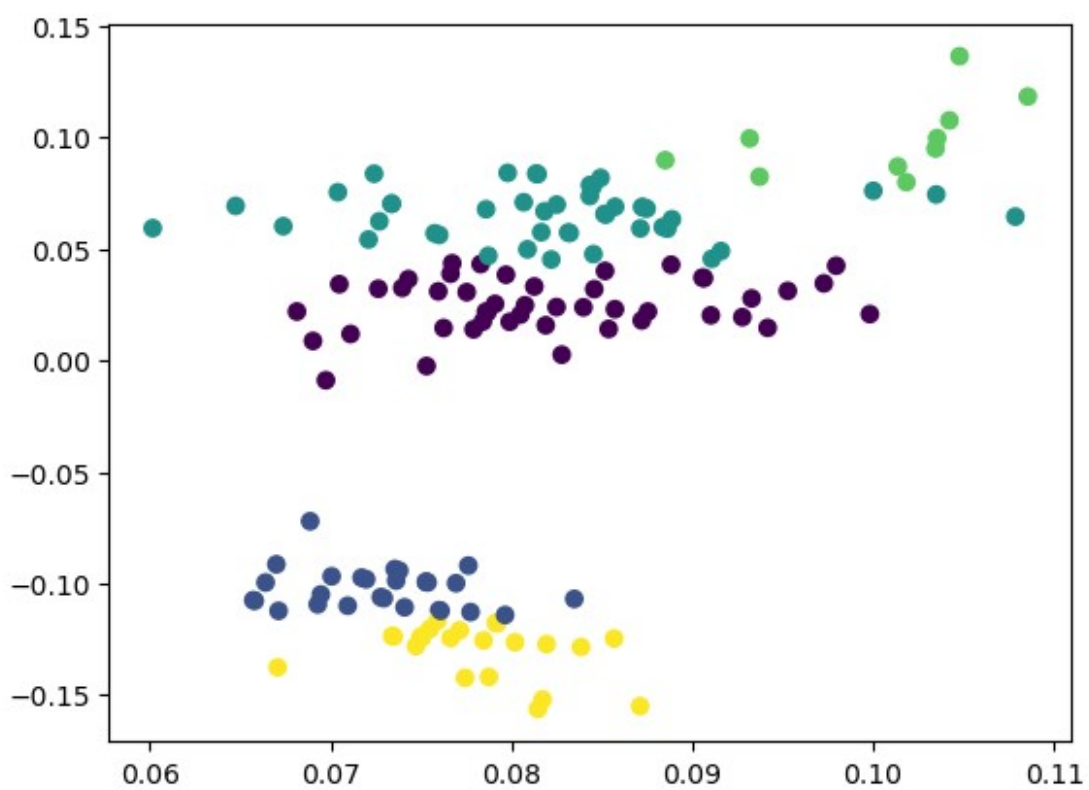
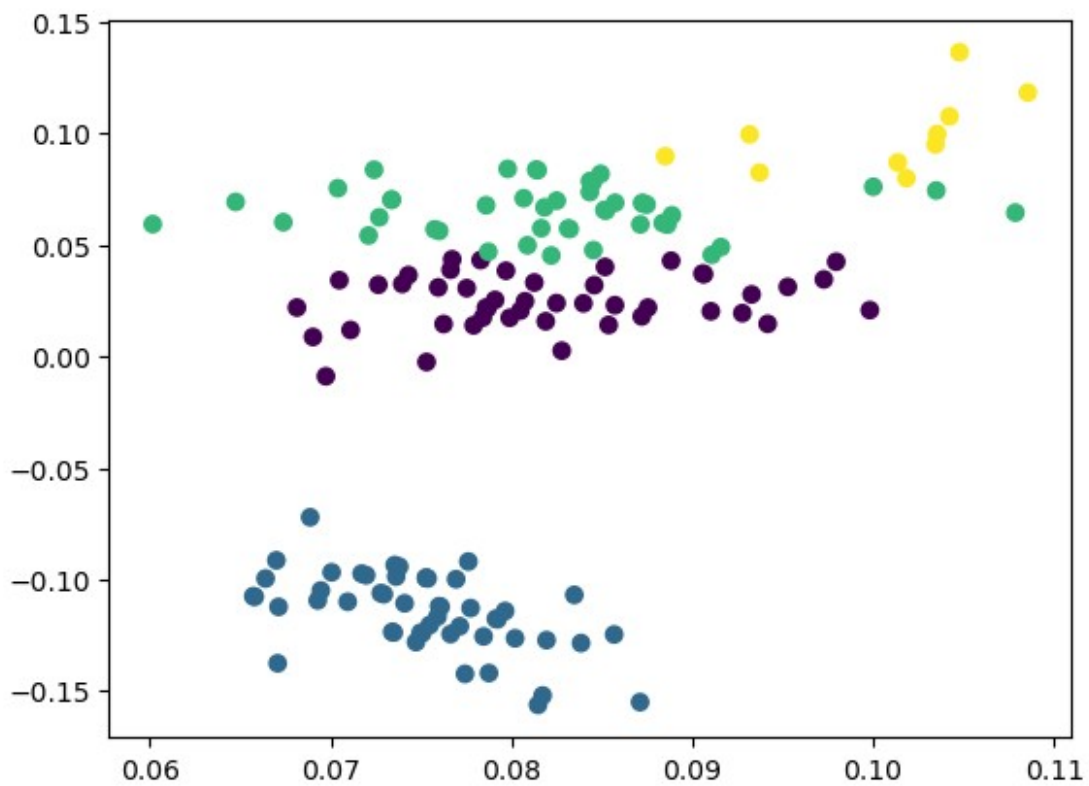
# Beep-Boop
accuracy_k3 = accuracy(y_hats[1], y)
print(f"Accuracy for when K=3: {accuracy_k3}, or {accuracy_k3 * 100}%")

# I don't really know how to check for K != 3 since I don't have the
data to compare with.
# I can only assume the correct solution, but only for K = 2,
otherwise I am uncertain myself.

Accuracy for when K=3: 0.9467, or 94.67%

```





<Figure size 640x480 with 0 Axes>

Comment: Looks nice. Almost correct. The accuracy is pretty high (approximately 94.67%) for $K = 3$, but not perfect. Although there is no correct labeling with other K values, we can easily assume one when $K = 2$, and it looks flawless. For $K = 4+$ I can't tell for sure myself to be honest, but it looks... decent?

Part 2.2: Quantitative Assessment of K-Means

Above, we used k-means for clustering and assessed the results qualitatively by visualizing them. However, we often want to measure in a quantitative way how good a clustering is. To do this, we will use a classification task to **evaluate numerically how good the learned clusters are for all the different values of k you used above (2 to 5)**.

Informally, our evaluation will work as follows: For each of our clusterings ($k=2$ to $k=5$), we will try to learn a mapping from the identified clusters to the correct labels of our datapoints. The reason this can be a sensible evaluation, is that to learn a good mapping, we have to have identified clusters that correspond to the actual classes in our data. We will in other words train 4 different classification models (one for each k value), where the input to our classifier is the cluster each datapoint belongs to, and the target is the correct class for this datapoint. In other words, **we aim to learn to classify datapoints as well as possible with the only information available to the classifier being the cluster that datapoint belongs to**. For some values of k , we will get poorly performing classifiers, indicating that this clustering has not revealed the correct class division in our data.

In practice, you will do the following: Reload the *iris* dataset. Import a standard `LogisticRegression` classifier from the module `sklearn.linear_model`. Use the k-means representations learned previously (`yhats[2], ..., yhats[5]`) and the true label to train the classifier. Evaluate your model on the training data (we do not have a test set, so this procedure will assess the model fit instead of generalization) using the `accuracy_score()` function from the `sklearn.metrics` module. Plot a graph showing how the accuracy score varies when changing the value of k . Comment on the results.

- Train a Logistic regression model using the first two dimensions of the PCA of the iris data set as input, and the true classes as targets.
- Report the model fit/accuracy on the training set.
- For each value of K :
 - One-Hot-Encode the classes outputed by the K-means algorithm.
 - Train a Logistic regression model on the K-means classes as input vs the real classes as targets.
 - Calculate model fit/accuracy vs. value of K .
- Plot your results in a graph and comment on the K-means fit.

```
from sklearn.linear_model import LogisticRegression
from sklearn import metrics

# Please ignore the warnings here too

# Declarations
y_hats = []
accuracies = []
```



```

# Copy-pasted this segment
for i in range(len(k_values)):
    kmeans = KMeans(n_clusters=k_values[i], random_state=seed)
    y_hat = kmeans.fit_predict(projection)

    # Onehot encoding some labels in a very fancy way
    # Aka turning for example 3 out of 0-5 into (0, 0, 0, 1, 0, 0)
    y_hot = np.zeros((y_hat.shape[0], max(y_hat) + 1)) # Making zeros
    y_hot[np.arange(y_hat.shape[0]), y_hat] = 1 # Changing
selected value to 1

    # Doing some logistic regression to map clusters to actual classes
    # Pretty straightforward to be honest
    clf = LogisticRegression()
    clf.fit(y_hot, y)

    # Making some predictions and calculating accuracy
    acc = metrics.accuracy_score(y, clf.predict(y_hot)) # Why not use
my function? ;(
    acc = round(acc, 4) # Rounding to
4 decimals, standard practice
    accuracies.append(acc) # Adding to
list

    # Printing some results
    print(f"K={k_values[i]}: {acc}, or {acc * 100}%")

# Plotting the results in a nice way
plt.plot(k_values, accuracies, marker='o')
plt.plot(k_values[1], accuracy_k3, marker='o')
plt.title('Logistic Regression Accuracy vs K in K-Means')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Accuracy')
plt.grid(True)
plt.show()

# P.S. I get how the accuracy works now, but that is kinda dumb :/
# But who am I to complain; It produces miracles :D

```

```

K=2: 0.6667, or 66.67%
K=3: 0.9467, or 94.67%
K=4: 0.96, or 96.0%
K=5: 0.96, or 96.0%

```

```

C:\Users\Justa\anaconda3\envs\in3050\Lib\site-packages\sklearn\
cluster\_kmeans.py:1429: UserWarning: KMeans is known to have a memory
leak on Windows with MKL, when there are less chunks than available
threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=1.

```


Conclusions

In this notebook we studied **unsupervised learning** considering two important and representative algorithms: **PCA** and **k-means**.

First, we implemented the PCA algorithm step by step; we then ran the algorithm on synthetic data in order to see its working and evaluated when it makes sense to use it and when it doesn't. We then considered two typical uses of PCA: for **visualization** on the *iris* dataset, and for **compression-decompression** on the *lfw* dataset. We also looked at an additional question that arises when using PCA: the problem of **selection of hyper-parameters**, that is, how to select the optimal hyper-parameter of our algorithm for a particular task.

We then moved to consider the k-means algorithm. In this case we used the implementation provided by *scikit-learn* and applied it to another prototypical unsupervised learning problem: **clustering**. We processed the *iris* dataset with *k-means* and evaluated the results visually. We also considered the problem of **quantitative evaluation** of the results, that is, how to measure the performance or usefulness of k-means clustering on a downstream task (classifying the *iris* samples into their species).